

Formal Modelling and Analysis of a Self-Adaptive Robotic System

Juliane Päßler¹ (✉) , Maurice H. ter Beek² , Ferruccio Damiani³ ,
S. Lizeth Tapia Tarifa¹ , and Einar Broch Johnsen¹ 

¹ University of Oslo, Oslo, Norway

`{julipas,sltarifa,einarj}@ifi.uio.no`

² ISTI-CNR, Pisa, Italy

`maurice.terbeek@isti.cnr.it`

³ University of Turin, Turin, Italy

`ferruccio.damiani@unito.it`

Abstract. Self-adaptation is a crucial feature of autonomous systems that must cope with uncertainties in, e.g., their environment and their internal state. Self-adaptive systems are often modelled as two-layered systems with a *managed* subsystem handling the domain concerns and a *managing* subsystem implementing the adaptation logic. We consider a case study of a self-adaptive robotic system; more concretely, an autonomous underwater vehicle (AUV) used for pipeline inspection. In this paper, we model and analyse it with the feature-aware probabilistic model checker ProFeat. The functionalities of the AUV are modelled in a feature model, capturing the AUV’s variability. This allows us to model the managed subsystem of the AUV as a family of systems, where each family member corresponds to a valid feature configuration of the AUV. The managing subsystem of the AUV is modelled as a control layer capable of dynamically switching between such valid feature configurations, depending both on environmental and internal conditions. We use this model to analyse probabilistic reward and safety properties for the AUV.

Keywords: feature models · probabilistic model checking · self-adaptive systems · cyber-physical systems · robotics.

1 Introduction

Many software systems are subject to different forms of uncertainty like changes in the surrounding environment, internal failures and varying user requirements. Often, manually maintaining and adapting these systems during runtime by a system operator is prohibitively expensive and error-prone. Enabling systems to adapt themselves provides several advantages. A system that is able to perform self-adaptation can also be deployed in environments where, e.g., communication between an operator and the system is very limited or impossible, like in space or under water. Thus, self-adaptation gives a system a higher level of autonomy.

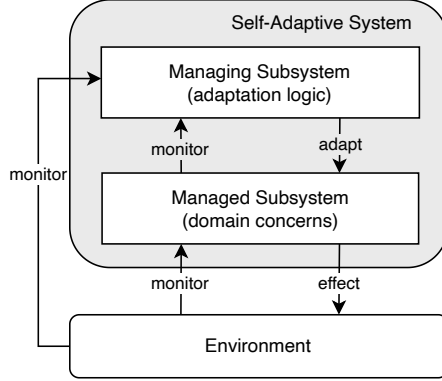


Fig. 1: Two-level SAS architecture

A self-adaptive system (SAS) can be implemented using a two-layered approach which decomposes the system into a *managed* and a *managing* subsystem [18], see Fig. 1. The *managed* subsystem deals with the domain concerns and tries to reach the goals set by the system’s user, e.g., navigating a robot to a specific location. The *managing* subsystem handles the adaptation concerns and defines an adaptation logic that specifies a strategy on how the system can fulfil the goals under uncertainty [25], e.g., adapting to changing environmental

conditions. While the managed subsystem may affect the environment via its actions, the managing subsystem monitors the environment and the internal state of the managed subsystem. By using the adaptation logic, the managing subsystem deduces whether and which reconfiguration is needed and adapts the managed subsystem accordingly.

This paper models and analyses the case study of a self-adaptive autonomous underwater vehicle (AUV) as a two-layered system based on Markov decision processes. The functionalities of the managed subsystem of the AUV are modelled in a feature model, making the dependencies and requirements between the components of the AUV explicit. The behaviour of the managed subsystem is modelled as a probabilistic transition system whose transitions may be equipped with feature guards, which only allow a transition to be taken if the feature guarding it is included in the current system configuration. Thus, it is modelled as a family of systems whose family members correspond to valid feature configurations. As the behaviour of the AUV depends on environmental and internal conditions, which are both hard to control, we opted for a probabilistic model in which uncontrolled events, like a thruster failure, occur with given probabilities. We model the behaviour of the managing subsystem as a control layer that switches between the feature configurations of the managed subsystem according to input from the probabilistic environment model and the managed subsystem. We consider a simplified version of an AUV, with limited features and variability, but there are many different possibilities to extend the model to a more realistic underwater robot.

The case study is modelled in ProFeat [8], a tool for probabilistic family-based model checking. Family-based model checking provides a means to simultaneously model check, in a single run, properties of a family of models, each representing a different configuration [23]. Analyses with ProFeat give system operators an estimate of mission duration and the AUV’s energy consumption, as well as some safety guarantees.

The main contributions of this paper are as follows:

- A case study of an SAS from the underwater robotics domain, modelled as a probabilistic feature guarded transition system with dynamic feature switching;
- Automated verification of (quantitative) properties that are important for roboticists, using family-based analysis.

Outline. Sec. 2 presents the case study of pipeline inspection with an AUV. Sec. 3 explains both the behaviour of the managed and managing subsystem of the AUV and the environment, as well as their implementation in ProFeat. Sec. 4 presents quantitative analyses conducted on the case study. Sec. 5 provides related work. Sec. 6 discusses our results and ideas for future work.

2 Case Study: Pipeline Inspection by AUV

In this section, we introduce our case study of an AUV used for pipeline inspection, which was inspired by the exemplar SUAVE [22].

An AUV has the mission to first find and then inspect a pipeline located on a seabed. During system operation, the water visibility (i.e., the distance in meters within which the AUV can perceive objects) might change (e.g., due to currents that swirl up the seabed), while one or more of the AUV's thrusters might fail and needs to be restarted before the mission can be continued.

The AUV can choose to operate at three different altitudes, *low*, *med* (for medium) and *high*. A higher altitude allows the AUV to have a wider field of view and thus increases its chances of finding the pipeline during its search. The probability of a thruster failure is lower at a higher altitude because, e.g., seaweed might wrap around the thrusters at a lower altitude. However, the altitude at which the AUV can perceive the seabed depends on the water visibility. With low water visibility, the AUV cannot perceive the seabed from a high or medium altitude. Thus, it is not always possible to operate at a high or medium altitude, and the altitude of the AUV needs to be changed during the search, depending on the current environmental conditions. Once the pipeline is found, the AUV will follow it at a low altitude to avoid costs for switching altitudes. In fact, once found, a wider field of view provides no benefit. However, the AUV can also lose the pipeline again (e.g., when the pipeline was partly covered by sand or the AUV's thrusters failed for some time causing the AUV to drift off its path). In this case, the AUV has to search the pipeline again, enabling all three altitudes.

Two-layered View of the AUV. Considering the AUV as a two-layered SAS, the AUV's managed subsystem is responsible for the search for and inspection of the pipeline. Depending on the current task and altitude of the AUV, a different configuration of the managed subsystem must be chosen. Thus, the managed subsystem can be seen as a family of systems where each family member corresponds to a valid configuration of the AUV. To do so, the different altitudes for navigation (*low*, *med* and *high*) and the tasks *search* and *follow* can be seen as *features* of the managed subsystem that adhere to the feature model in Fig. 2,

which models the dependencies and constraints among the features. Each configuration of the AUV contains exactly one feature for navigation and one for pipeline inspection, and feature *follow* requires feature *low*, yielding four different configurations of the managed subsystem of the AUV.

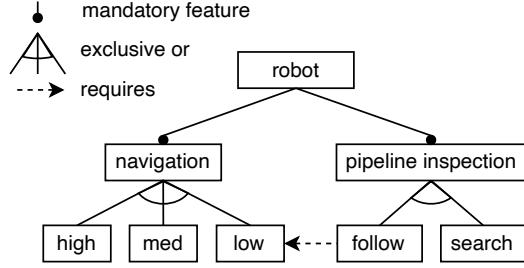


Fig. 2: Feature model of the case study

according to the current water visibility. If the water visibility is good, all three features can be activated; if the water visibility is average, *high* cannot be activated; and if the water visibility is poor, only *low* can be activated. The managing subsystem switches from the feature *search* to *follow* if the pipeline was found, and from *follow* to *search* if the pipeline was lost.

The managing subsystem of the case study switches between these configurations during runtime by activating and deactivating the sub-features of *navigation* and *pipeline inspection*, while the resulting feature configuration has to adhere to the feature model in Fig. 2. The features *low*, *med* and *high* are activated and deactivated ac-

3 Modelling the AUV Case Study with ProFeat

In this section, we describe the behavioural model of the managed and managing subsystem and the environment and model the case study with the family-based model checker ProFeat¹ [8]. ProFeat provides a means to both specify probabilistic system families and perform family-based quantitative analysis on them. It extends the probabilistic model checker PRISM² [19] with functionalities such as family models, features and feature switches. Thereby, it enables family-based modelling and (quantitative) analysis of probabilistic systems in which feature configurations may dynamically change during runtime. The whole model can be analysed with probabilistic family-based model checking using PRISM. The probabilities used in our model are estimates and have not been validated by experiments, since in this paper our goal was not to make a model that is as realistic as possible, but rather to show the feasibility of our approach.

Similar to an SAS, a ProFeat model can be seen as a two-layered model, as illustrated in Fig. 1. The behaviour of a family of systems that differ in their features, such as the managed subsystem of an SAS, can be specified. Then a so-called *feature controller* can activate and deactivate the features during runtime, and thus change the behaviour of the system, such as the managing subsystem of an SAS that changes the configuration of the managed subsystem. Furthermore, the environment can be specified as a separate module that interacts with the

¹ <https://pchrzson.github.io/profeat>.

² <https://www.prismmodelchecker.org/manual>

managed and managing subsystem. Thus, ProFeat is well suited to model and analyse the case study described in Sec. 2.

A ProFeat model consists of three parts: an obligatory feature model that specifies features and their relations and constraints, obligatory modules that specify the behaviour of the features, and an optional feature controller that activates or deactivates features. The pipeline inspection case study was modelled as a Markov decision process in ProFeat.³ It consists of (i) the implementation of the feature model of Fig. 2; (ii) modules describing the behaviour of the managed subsystem of the AUV (see Fig. 3) and of the environment (see Fig. 4); and (iii) the feature controller that switches between features during runtime, corresponding to the managing subsystem of the AUV (see Fig. 5).

We start by explaining how the feature model was implemented in ProFeat in Sec. 3.1, then describe the behaviour and implementation of the managed and managing subsystem and of the environment in Sec. 3.2, 3.4, and 3.3 respectively.

3.1 The Feature Model

We first show how the feature model of the case study is expressed in ProFeat, including connections and constraints among features. Each feature is specified within a **feature** ... **endfeature** block, the declaration of the root feature is done in a **root feature** ... **endfeature** block.

The Root Feature. An excerpt of the implementation of the root feature of the pipeline inspection case study according to Fig. 2 is displayed in Listing 1.1. The root feature can be decomposed into subfeatures; in this case only one, the subfeature **robot**, see Line 2. The **all of** keyword indicates that all subfeatures have to be included in the feature configuration if the parent feature, in this case the root feature, is included. It is, e.g., also possible to use the **one of** keyword if exactly one subfeature has to be included, see Line 2 of Listing 1.2. The modules modelling the behaviour of the root feature are specified after the keyword **modules**. In this case study, the root feature is the only feature specifying modules, thus the behaviour of all features is modelled in the modules **auv** and **environment** described later.

Contrary to an ordinary feature model, ProFeat allows to specify feature-specific rewards in the declaration of a feature. Like costs, rewards are real values, but unlike costs (and although they may be interpreted as costs) rewards are meant to motivate rather than penalise the execution of transitions. Each reward is encapsulated in a **rewards** ... **endrewards** block. In the case study, we consider the rewards *time* and *energy*, see Lines 4–18 of Listing 1.1. During each transition the AUV module takes, the reward *time* is increased by 1; it is a transition-based reward, see Line 5. We assume that one time step corresponds to one minute, allowing us to compute an estimate of a mission’s duration.

The reward *energy* is a state-based reward and can be used to estimate the necessary battery level for a mission completion. If a thruster of the AUV failed

³ The model is publicly available at [21].

```

1 root feature
2   all of robot;
3   modules auv, environment;
4   rewards "time"
5     [step] true : 1;
6   endrewards
7   rewards "energy"
8     // Costs for being in a recovery state
9     (s=recover_high) : 2;
10    // .. omitted code ..
11
12    // Costs for switching altitudes
13    (s=search_high) & active(low) : 4;
14    (s=search_high) & active(med) : 2;
15    (s=found) & active(high) : 4;
16    (s=found) & active(med) : 2;
17    // .. omitted code ..
18  endrewards
19 endfeature

```

Listing 1.1: An excerpt of the declaration of the root feature of the case study

```

1 feature navigation
2   one of low, med, high;
3   initial constraint active(low);
4 endfeature

```

Listing 1.2: The declaration of the navigation feature of the case study

and needs to be recovered, a reward of 2 is given, see, e.g., Line 9. The model also reflects that switching between the search altitudes requires significant energy. Since the altitude is switched if the AUV is in a search state and a navigation subfeature that does not correspond to the current search altitude is active, a higher energy reward is given in these states. If the AUV needs to switch between low and high altitude, as, e.g., in Line 13, an energy reward of 4 is given, while all other altitude switches receive a reward of 2, see, e.g., Line 14. Since the altitude must be changed to *low* once the pipeline is found, these cases also receive an energy reward as explained above, see Lines 15–16. All other states receive an energy reward of 1. We use the function `active` to determine which feature is active, i.e., included in the current feature configuration; given a feature, the function returns true if it is active and false otherwise. Note that both time and energy rewards are interpreted as costs.

Ordinary Features. The remainder of the feature model is implemented similar to the root feature, but the features do not contain feature-specific modules or rewards. The features are implemented and named according to the feature model in Fig. 2. To have only one initial state, we initialise the model with the features `search` and `low` active, using the keyword `initial constraint`, see Line 3 of Listing 1.2. As an example of the implementation of another feature, the declaration of the feature `navigation` can be seen in Listing 1.2.

3.2 The Managed Subsystem

The Behavioural Model of the Managed Subsystem. The behaviour of the managed subsystem of the AUV can be described by a probabilistic transition system

equipped with features that guard transitions (a probabilistic featured transition system). Only if the feature guarding a transition is included in the current configuration of the managed subsystem of the AUV, the transition can be taken. This transition system adheres to the feature model in Fig. 2 and is depicted in Fig. 3, where a number of details have been omitted to avoid cluttering (in particular, all probabilities). The details can be obtained from the publicly available model in [21]. The probabilistic model allows to easily model the possibilities of, e.g., finding and losing the pipeline depending on the system configuration.

The transition system can roughly be divided into two parts, one concerning the search for and one the following of the pipeline, as shown by the grey boxes in Fig. 3. At deployment time, i.e., in state *start task*, the AUV can either immediately start following the pipeline if it was deployed above it, or start searching for it. During the search for the pipeline, i.e., when the AUV is in the grey area labelled *search*, the feature *search* should be active and remain active until the state *found* is reached. The managing subsystem can switch between the features *low*, *med* and *high* during every transition, depending on the water visibility as described in Sec. 2. Once the pipeline is found, the managing subsystem has to deactivate the feature *search* and activate the feature *follow*, which also implies activating the feature *low* and deactivating *med* and *high* due to the feature constraints in Fig. 2. We assume that the managing subsystem activates and deactivates features during transitions, so the features *follow* and *low* should be activated during the transition from the state *found* to the state *start task*. When the AUV is following the pipeline, i.e., in the grey area labelled *follow*, it can also lose the pipeline again, e.g., because of sand covering it or because it drifted off its path due to thruster failures. Then the managing subsystem has to activate the feature *search* during the transition from *lost pipe* to *start task*.

We distinguish two kinds of transitions: transitions that model the behaviour of a certain configuration of the managed subsystem (black transitions) and (featured) transitions that switch between configurations, enabled by the managing subsystem during runtime (blue transitions). The labels *search*, *follow*, *low*, *med* and *high* on the transitions represent the features that have to be active to execute the respective transition. The transitions between configurations (blue) implicitly carry the action to start the task or go to the altitude specified by the feature associated with the transition. For instance, the transitions from *search low* to *search medium* can be taken if the feature *med* is active because the transition has the guard *med*. When taking this transition, the AUV should perform the action of going to a medium altitude. The transitions inside a configuration (black) with a feature label contain the implicit action to stay at the current altitude because the navigation subfeature has not been changed during the previous transition.

Whether a transition inside the configuration or between configurations is executed in the search states *search low*, *search medium* and *search high* depends on the managing subsystem, i.e., the controller switching between features (see Sec. 3.4). If the managing subsystem switched between the features *low*, *med* and *high* during the last transition, a transition to the search state corresponding

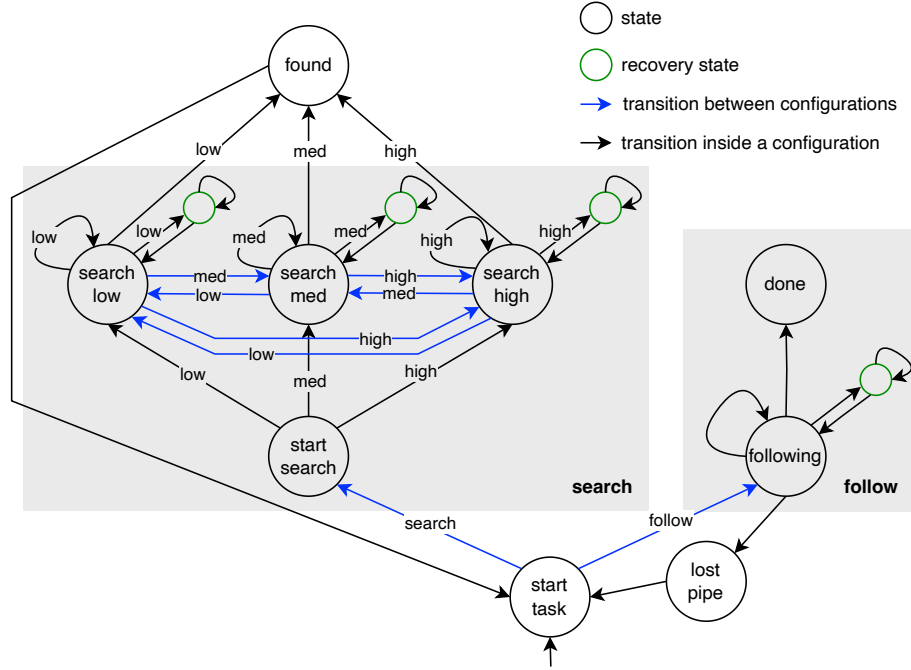


Fig. 3: The managed subsystem of the AUV

to the new feature will be executed, i.e., the configuration will be changed. Otherwise, a transition inside the configuration will be executed. For instance, consider the state *search low*. If the feature *low* is active, then a black transition will be executed. If, however, the managing subsystem deactivated the feature *low* during the last transition and activated either *med* or *high*, then the AUV will perform a transition to the state *search medium* or *search high*, respectively.

The ProFeat Implementation of the Managed Subsystem. The module `auv` models the behaviour of the managed subsystem of the AUV as displayed in Fig. 3, see Listing 1.3 for an excerpt of the model. As in Fig. 3, there are thirteen enumerated states in the ProFeat module with names that correspond to the state labels in the figure. The recovery states are named according to the state they are connected to (e.g., the recovery state connected to `search_high` is called `recover_high`). The variable `s` in Line 2 represents the current state of the AUV and is initialised using the keyword `init` with the state `start_task`. To record how many meters of the pipeline have already been inspected, the variable `d_insp` in Line 3 represents the distance the AUV has already inspected the pipeline, it is initialised with 0. The variable `inspect` represents the desired inspection length and can be set by the user during design time. Since the number of times a thruster failed impacts how much the AUV deviates from its path, the variable `t_failed` can be increased if a thruster fails while the AUV follows the pipeline.

```

1 module auv
2   s : [0..12] init start_task;
3   d_insp : [0..inspect] init 0;
4   t_failed : [0..infl_tf] init 0;
5
6   // To the correct task
7   [step] (s=start_task & active(search)) -> 1: (s'=start_search);
8   [step] (s=start_task & active(follow)) -> 1: (s'=following);
9
10  // .. omitted code ..
11  // From search state to another state
12  [step] (s=search_high & active(high))
13    -> 0.59:(s'=found)
14    + 0.4:(s'=search_high)
15    + 0.01:(s'=recover_high);
16  [step] (s=search_high & active(med)) -> 1:(s'=search_med);
17  [step] (s=search_high & active(low)) -> 1:(s'=search_low);
18  // .. omitted code ..
19
20  // Go to other task if pipeline is found
21  [step] (s=found) -> 1:(s'=start_task);
22
23  // Following the pipeline
24  [step] (s=following) & (d_insp<inspect) & (t_failed=0)
25    -> 0.92: (s'=following) & (d_insp'=d_insp+1)
26    + 0.05: (s'=lost_pipe)
27    + 0.03:(s'=recover_following)
28    & (t_failed'=(t_failed<infl_tf? t_failed+1 : t_failed));
29  [step] (s=following) & (d_insp<inspect) & (t_failed>0)
30    -> 0.92*(1-t_failed/infl_tf): (s'=following)
31    & (d_insp'=d_insp+1) & (t_failed'=t_failed-1)
32    + 0.05*(1+((0.92*t_failed)/(0.05*infl_tf))): (s'=lost_pipe)
33    + 0.03:(s'=recover_following)
34    & (t_failed'=(t_failed<infl_tf? t_failed+1 : t_failed));
35  [step] (s=following) & (d_insp=inspect) -> (s'=done);
36
37  // Lost the pipeline
38  [step] (s=lost_pipe) -> 1: (s'=start_task) & (t_failed'=0);
39
40  // Recovery states
41  [step] (s=recover_high) -> 0.5:(s'=recover_high) + 0.5:(s'=search_high);
42  // .. omitted code ..
43 endmodule

```

Listing 1.3: An excerpt of the ProFeat AUV module of the case study

It is bounded by the influence a thruster failure can have on the system (`infl_tf`) that can be set by the user during design time.

The behaviour of the module is specified with *guarded commands*, corresponding to possible, probabilistic transitions, of the following form.

$$[\text{action}] \text{ guard} \rightarrow \text{prob_1: update_1} + \dots + \text{prob_n: update_n};$$

A command may have an optional label `action` to annotate it or to synchronise with other modules. In PRISM, the `guard` is a predicate over global and local variables of the model, which can also come from other modules. ProFeat extends the guards by, e.g., enabling the use of the function `active`. If the guard is true, then the system state is changed with probability `prob_i` using `update_i` for all i . An update describes how the system should perform a transition by giving new values for variables, either directly or as a function using other variables.

For instance, consider the command in Lines 12–15, which can be read as follows. If the system is in state `search_high` and the feature `high` is active, then with a probability of 0.59, the system changes its state to `found`, with a probability of 0.4 it changes to `search_high` and with a probability of 0.01 it changes to `recover_high`. These are exactly the black transitions shown in Fig. 3 exiting from state *search_high*. This command also has an action label, `step`. Using this action label, it synchronises with the environment module and the feature controller, as described later. The blue transitions exiting state *search_high* in Fig. 3 are modelled in Lines 16–17. If the model is in state `search_high`, but the feature `low` or `med` is active, indicating that the AUV should go to the respective altitude, then the state is changed to the respective search state. The transitions exiting the states `search_med` and `search_low` are modelled similarly. However, the probability of going to the state `found` is highest from state `search_high` and lowest from `search_low` because the AUV has a wider field of view when performing the search at a higher altitude. Furthermore, the probability of a thruster failure, i.e., of going to the respective `recover` state, is highest in state `search_low` and lowest in state `search_high` because the probability of seaweed getting stuck in the thrusters is higher at a lower altitude. If the AUV found the pipeline, then a transition to `start_task` is taken, see Line 21.

From the state `start_task`, a transition to either `start_search` or `following` can be taken, depending on which subfeature of `pipeline_inspection` is currently active, see Lines 7–8.

From the `following` state, the transitions that can be taken depend on the variables `d_insp` and `t_failed`. Lines 24–28 consider the case where the distance of the pipeline that has already been inspected (`d_insp`) is less than the distance the pipeline should be inspected (`inspect`) and the variable `t_failed` is 0, indicating that there were no recent thruster failures. Then the AUV stays in the `following` state and inspects the pipeline one more meter, it loses the pipeline, or a thruster fails and it transitions to the failure state and increases `t_failed` if `t_failed` is not at its maximum. Lines 29–34 consider the case where `d_insp` is less than `inspect` and `t_failed` is greater than 0. In this case, the probabilities of following and of losing the pipeline depend on the value of `t_failed`. The bigger the value, the more likely it is to lose the pipeline because it indicates that the AUV’s thrusters did not work for some time, causing it to drift off its path. If the already inspected distance is equal to the required inspection distance, the AUV transitions to the `done` state (see Line 35) and finishes the pipeline inspection. If the AUV lost the pipeline (see Line 38), then a transition to `start_task` is taken and the variable `t_failed` is set to 0 again.

When the AUV is in a recovery state, it can either stay there for another time step or exit it again to the state from where the recovery was triggered (see Line 41).

All commands in the module `auv` are labelled with `step`. Thus, every transition receives a time reward of 1, i.e., the time advances with every transition the AUV takes, see Lines 4–6 of Listing 1.1.

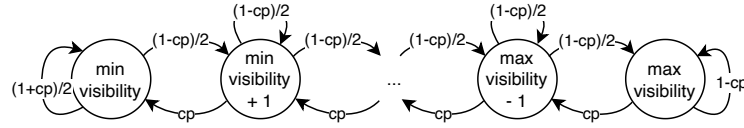


Fig. 4: The behaviour of the environment

```

1 module environment
2   water_visib : [min_visib..max_visib]
3   init round((max_visib-min_visib)/2);
4   [step] true -> current_prob: (water_visib' = (water_visib=min_visib?
5     min_visib:water_visib-1)) + (1-current_prob)/2: (water_visib' =
6     (water_visib=max_visib? max_visib:water_visib+1))
7     + (1-current_prob)/2: true;
8 endmodule

```

Listing 1.4: The ProFeat environment module of the case study

3.3 The Environment

The Behavioural Model of the Environment. We assume that there is a minimum and a maximum visibility of the environment, depending on where the AUV is deployed and set by the user during design time. Furthermore, different environments also have different probabilities of currents that influence the water visibility. This can also be set during design time. The behaviour of the environment is then modelled as depicted in Fig. 4, where cp represents the *current probability*. With the probability of currents cp , the water visibility decreases by 1, while it stays the same or increases by 1 with probability $(1-cp)/2$. If the water visibility is already at minimum visibility, the water visibility stays the same with probability $(1+cp)/2$ and, at maximum visibility, it stays the same with probability $(1-cp)$.

The Implementation of the Environment in ProFeat. The environment is modelled in a separate environment module, see Listing 1.4. The variable `water_visib` in Line 2 reflects the current water visibility and is initialised parametrically, depending on the minimum and maximum visibility, see Line 3. The function `round()` is pre-implemented in the PRISM language and rounds to the nearest integer. The environment module synchronises with the AUV module via the label of its action, `step`. Since the guard of the only action in the environment module is `true`, the environment executes a transition every time the AUV module does. By decoupling the environment module from the AUV module, we obtain a separation of concerns which makes it easier to change the model of the environment if needed.

3.4 The Managing Subsystem

The Behavioural Model of the Managing Subsystem. As described in Sec. 2, the managing subsystem of the AUV implements the AUV's adaptation logic, which corresponds to activating and deactivating the features of the managed subsystem. The behaviour of the managing subsystem of the AUV is displayed

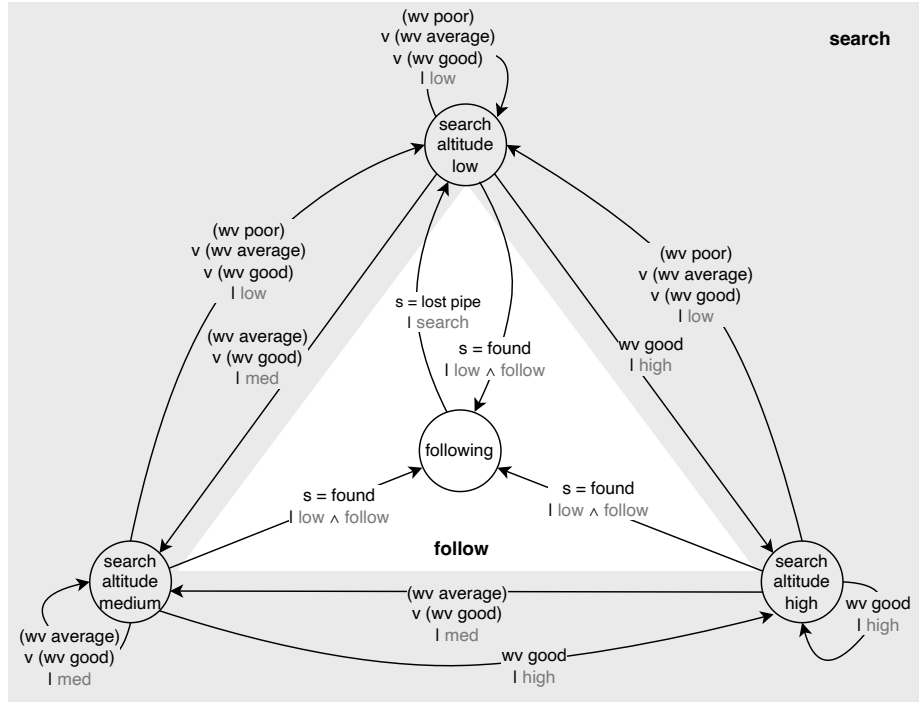


Fig. 5: The managing subsystem of the AUV

in Fig. 5. The grey area of the figure includes the transitions that can be taken during the search for the pipeline, and the white area the transitions once the pipeline has been found. Each transition contains a guard, written in black, and an action, written in grey after a vertical bar.

During the search for the pipeline, i.e., in the grey area of Fig. 5, the managing subsystem activates and deactivates the features *low*, *med* and *high* according to the current water visibility as described in Sec. 2. The activated feature is displayed in grey on the transition, implicitly the other two subfeatures of *navigation* are deactivated. Note that the transitions in the grey area implicitly carry the guard $s \neq \text{found}$, i.e., the AUV is not in the state *found*, because they represent the transitions during the search for the pipeline. This guard was omitted for better readability.

Once the pipeline has been found, i.e., the managed subsystem is in the state *found*, one of the transitions in the white area, guarded by $s = \text{found}$, is taken. These transitions include the action of activating *low* and *follow*, and thus deactivating *med*, *high* and *search*. When the AUV loses the pipeline, i.e., it is in the state *lost pipe*, the managing subsystem activates *search* and deactivates *follow*. Since the AUV is following the pipeline at a low altitude, the AUV will start searching at a low altitude.

```

1 formula med_visib = (max_visib-min_visib)/3;
2 formula high_visib = 2*(max_visib-min_visib)/3;
3
4 controller
5   // Change altitude depending on water visibility
6   [step] (s!=found) & active(search) & water_visib < med_visib
7     -> activate(low) & deactivate(high) & deactivate(med);
8   [step] (s!=found) & active(search)
9     & med_visib <= water_visib & water_visib < high_visib
10    -> activate(low) & deactivate(med) & deactivate(high);
11   [step] (s!=found) & active(search)
12     & med_visib <= water_visib & water_visib < high_visib
13    -> activate(med) & deactivate(low) & deactivate(high);
14   // .. omitted code ..
15
16   // Switch task from "search" to "follow"
17   [step] (s=found) & active(search)
18     -> deactivate(search) & activate(follow) & activate(low)
19       & deactivate(med) & deactivate(high);
20
21   // Switch task from "follow" to "search"
22   [step] (s=lost_pipe) & active(follow)
23     -> deactivate(follow) & activate(search);
24
25   // Enable transitions when following the pipeline
26   [step] (s!=lost_pipe) & active(follow) -> true;
27 endcontroller

```

Listing 1.5: An excerpt of the ProFeat feature controller of the case study

The Implementation of the Managing Subsystem in ProFeat. The managing subsystem of the AUV is implemented as a feature controller in ProFeat. The feature controller can also use *commands* to change the state of the system. Such commands are similar to those used in a module; they are mostly of the form [action] guard $\rightarrow$ update. Each command can have an optional label action to synchronise with the modules, and its guard is a predicate of global and local variables of the model and can also contain the function active. In contrast to the commands in the modules, the feature controller can activate and deactivate features in the update of a command. Several features can be activated and deactivated at the same time, but this cannot be done probabilistically and the resulting feature configuration has to adhere to the feature model.

In the pipeline inspection case study, subfeatures of navigation (i.e., the different altitudes at which the AUV can operate) and subfeatures of pipeline_inspection (i.e., the tasks the robot has to fulfil) can be switched by the feature controller during runtime, see Listing 1.5.

When the feature search is active and the pipeline has not been found yet, the feature controller activates and deactivates the altitudes non-deterministically, but according to the current water visibility, as described before. The minimum and maximum water visibility can be set by the user during design time and influence the altitudes associated with the features low, med and high; i.e., it influences when the feature controller is able to switch features. To reflect this, the variables med_visib and high_visib are declared as in Lines 1–2 (a formula in PRISM and ProFeat can be used to assign an identifier to an expression). If the water visibility is less than med_visib, the feature controller activates the feature low (see Lines 6–7) because the AUV cannot perceive the seabed from a higher

altitude. If the water visibility is between `med_visib` and `high_visib`, it chooses non-deterministically between `low` and `med` (see Lines 8–13), whereas it chooses non-deterministically between all three altitudes if the water visibility is above `high_visib`. Note that it is also possible to deactivate or activate a feature if it is already inactive or active, respectively.

When the pipeline is found, i.e., the AUV is in state `found`, the feature controller activates the feature `follow` and deactivates `search`, see Lines 17–19. Since the AUV should be at a low altitude while following the pipeline, the feature controller also deactivates the features `high` and `med` and activates `low`. If the AUV lost the pipeline, i.e., it is in state `lost_pipe`, the feature controller deactivates `follow` and activates `search` to start the search for the pipeline, see Lines 22–23.

The feature controller synchronises with the `auv` and `environment` modules via action label `step`. Since all transitions of the modules and feature controller have the same action label, they can only execute a transition if there is a transition with a guard evaluating to true in both modules and in the feature controller. Thus, the feature controller needs to include a transition doing nothing if the feature `follow` is active and the AUV is not in state `lost_pipe`, see Line 26.

4 Analysis

ProFeat automatically converts models to PRISM for probabilistic model checking. To analyse a PRISM model, properties can be specified in the PRISM property specification language, which includes several probabilistic temporal logics like PCTL, CSL and probabilistic LTL. For family-based analysis, ProFeat extends this specification language to include, e.g., the function `active`. (ProFeat constructs have to be specified in $\$ \{ \dots \}$ to be correctly translated to the PRISM property specification language.)

The operators used for analysis in this paper are `P` and `R`, which reason about probabilities of events and about expected rewards, respectively. Since we use Markov decision processes which involve non-determinism, these operators must be further specified to ask for the *minimum* or *maximum* probability and expected cost, respectively, for all possible resolutions of non-determinism.

The analysis of the model considered two different aspects. First, the rewards `energy` and `time` were used to compute some safety guarantees that can be used for the deployment of the AUV. Second, safety properties with regard to unsafe states were analysed. Note that it is not necessary to analyse whether the model satisfies the constraints of the feature model because this is automatically ensured by ProFeat. Of course, in addition to that, more complex analysis can be done. In this paper, we just give a taste of possible analyses to demonstrate the feasibility of our approach.

We analysed two different scenarios; the values used in these scenarios are reported in Table 1. Scenario 1 is in the North Sea, where the minimum and maximum water visibility (in 0.5 meter units) are relatively low and the probability of currents that decrease the water visibility is relatively high. In this case, only 10 meters of the pipeline have to be inspected. Scenario 2 is in the Caribbean

Table 1: Two different scenarios used for analysis

Scenario	min_visib	max_visib	current_prob	inspect
1 (North Sea)	1	10	0.6	10
2 (Caribbean Sea)	3	20	0.3	30

```

1 R{"energy"}min=? [F ${s=done}];
2 R{"energy"}max=? [F ${s=done}];

```

Listing 1.6: Analysis using the rewards

Sea, with a higher minimum and maximum visibility and a lower probability of currents compared to the North Sea, and 30 meters of pipeline that have to be inspected. For both scenarios, we first analysed whether it is always possible to finish the pipeline inspection, i.e., reach the state *done*. This could be confirmed since the minimum probability for all resolutions of non-determinism of eventually reaching the state *done* is 1.0.

Reward Properties. The rewards *time* and *energy* were used to analyse some safety properties related to the execution of the AUV. Since the AUV only has a limited amount of battery, an estimation of the energy needed to complete the mission is required. This ensures that the AUV is only deployed for the mission if it has sufficient battery to complete it. The commands in Listing 1.6 were used to compute the minimum and maximum expected energy (for all resolutions of non-determinism) to complete the mission. Since the model includes two reward structures, the name of the reward has to be specified in {"..."} after the R operator. Similarly, the minimum and maximum expected time to complete the mission was analysed to give the system operators an estimate of how long the mission might take. The results for Scenarios 1 and 2 are reported in Table 2. It can be seen that the variation of the parameters in the two scenarios strongly influences the expected energy and time of the mission. It is interesting to see that the difference between minimum and maximum expected energy and minimum and maximum expected time for Scenario 2 are significantly bigger than for Scenario 1. In particular, the maximum expected energy and time are much higher for Scenario 2 than for Scenario 1. Further analysis in this direction could investigate trade-offs between different scenarios and a better understanding of the influence in the results for the different parameters.

Table 2: Expected min-/maximum rewards for completing the mission for both scenarios

Scenario	Energy		Time	
	min	max	min	max
1	24.78	44.39	23.66	32.40
2	59.08	4723.29	55.54	1315.58

Unsafe States. Thruster failures, although we assume that they can be repaired, pose a threat to the AUV. Unforeseen events like strong currents might cause the AUV to be damaged, e.g., by causing it to crash into a rock. To analyse this, the state space was partitioned into two parts, *safe* and *unsafe* states. This was achieved by using labels, see Lines 1–4 of Listing 1.7.

```

1 label "unsafe" = s=recover_high | s=recover_med | s=recover_low
2 | s=recover_following;
3 label "safe" = s=start_task | s=lost_pipe | s=start_search | s=search_high
4 | s=search_med | s=search_low | s=found | s=following | s=done;
5 Pmin=? [G "safe"];
6 filter(min, Pmin=? [ F<=k "safe" ], "unsafe");
7 filter(max, Pmax=? [ F<=k "unsafe" ], "safe");
8 filter(avg, Pmax=? [ F<=k "unsafe" ], "safe");

```

Listing 1.7: Analysis of unsafe states

These labels were then used to calculate the probability of several properties. The minimum probability of only taking safe states (see Line 5) was shown to be 0.65 for Scenario 1 and 0.32 for Scenario 2. As expected, the probability of only taking safe states is higher for a shorter pipeline inspection. It is also important to ensure that a safe state will be reached from an unsafe state after a short period of time, as, e.g., in Line 6, where k is an integer. For every unsafe state, the minimum probability (for all possible resolutions of non-determinism) of reaching a safe state within k time steps is calculated. Then the minimum over all these probabilities is taken. Thus, it gives the minimum probability of reaching a safe state from an unsafe state in k time steps. PRISM experiments allow analysing this property automatically for a specified range of k . Using PRISM experiments, it was shown that in both scenarios the probability of reaching a safe state from an unsafe state is above 0.95 after 5 time steps and above 0.99 after 7 time steps.

The probability of going to an unsafe state from a safe state should be as small as possible. This is analysed with the properties in Lines 7–8. First, the maximum probability (over all possible resolutions of non-determinism) for reaching an unsafe state from a safe state is calculated, and then the maximum (or average) is taken. Again, PRISM experiments were used to analyse this, the plotted graphs for Scenarios 1 and 2 are displayed in Fig. 6. They show that the probability of reaching an unsafe state from a safe state increases with the number of considered time steps. Furthermore, the probability of reaching an unsafe state from a safe state stabilises much later and at a higher value in Scenario 2 than in Scenario 1. While the maximum probability of reaching an unsafe state from a safe state stabilises after about 42 time steps at ≈ 0.37 in Scenario 1, it stabilises after about 76 time steps at ≈ 0.69 in Scenario 2. Similar differences can be observed for the average probability.

5 Related Work

The analysis of behavioural requirements is often crucial when developing an SAS that operates in the uncertainty of a physical environment. These requirements often use quantitative metrics that change during runtime. Both rule-based and goal-based adaptation logics can be used to enable the SAS to meet its behavioural requirements. Many practitioners rely on formal methods to provide evidence for the system’s compliance with such requirements [26,20], but many different methods are used [15,1]. We consider related work for family-based modelling and analysis approaches.

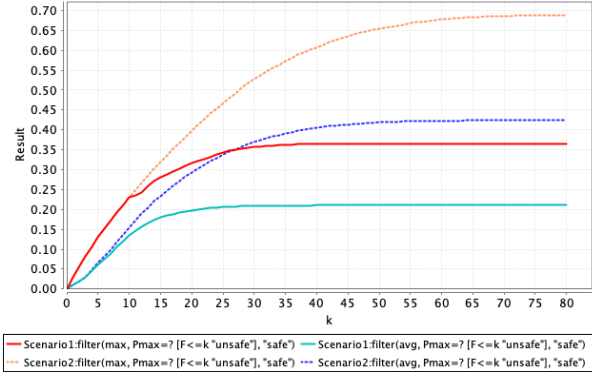


Fig. 6: Results for reaching an unsafe state from a safe state in k time steps

Family-based model checking of transition systems with features allows to model check properties of multiple behavioural models in a single run, following the seminal work by Classen et al. [10]. Such model-checking tools can be encoded in well-known classical model checkers like SPIN [17], NuSMV [9] or PRISM [19]. In this paper, we used ProFeat [8], a software tool built on top of PRISM for the analysis of feature-aware probabilistic models. Alternatively, QFLan [24] offers probabilistic simulations to yield statistical approximations, thus trading 100% precision for scalability. In [6,7], configurable systems are modelled and analysed as role-based systems, an extension of feature-oriented systems, with a focus on feature interaction; in contrast to our paper, they do not consider a separation between managed and managing subsystem.

Software product lines (SPLs) can be seen as families of (software product) models where feature selection yields variations in the products (configurations). SPLs have previously been proposed to model static variability, i.e., variability during design time, for robotic systems [12]. In [3] it is argued that most of the costs for robotic systems come from non-reusable software. A robotic system mostly contains software tailored to the specific application and embodiment of the robot, and often even software libraries for common robotic functionalities are not reusable. Therefore, they must be re-developed all the time. Thus, a new approach for the development of robotic software using SPLs is proposed in [3].

Finally, dynamic SPLs (DSPLs) [13,16] have been proposed to manage variability during runtime for self-adaptive robots [4]. There are several approaches that model, but do not analyse, SASs as DSPLs, e.g., [2,11,14]. For robotics, the authors in [12] propose the toolchain HyperFlex to model robotic systems as SPLs; it supports the design and reuse of reference architectures for robotic systems and was extended with the Robot Perception Specification Language for robotic perception systems in [5]. It allows to represent variability at different abstraction levels, and feature models from different parts of the system can be composed in several different ways. However, contrary to the approach used in this paper, HyperFlex only considers design time variability. Furthermore, it is only used for modelling robotic systems, not for analysing them.

6 Discussion and Future Work

In this paper, we used a feature model together with a probabilistic, feature guarded transition system to model the managed subsystem of an AUV used for pipeline inspection, and a controller switching between these features to model the managing subsystem of the AUV. This allowed modelling the managed subsystem of the AUV as a family of systems, where each family member corresponds to a valid feature configuration of the AUV. The managing subsystem could then be considered as a control layer capable of dynamically switching between these feature configurations depending on both environmental and internal conditions. The tool ProFeat was used for probabilistic family-based model checking, analysing reward and safety properties.

ProFeat allowed to model the two different layers of abstraction of an SAS, the managed and managing subsystem, which also makes it easier to understand the model and the adaptation logic. Furthermore, it makes analysing all configurations of the managed subsystem more efficient by enabling family-based model checking. However, it remains to be seen how this scales with larger models. We are unaware of other work that exploits the family-based modelling and analysis capabilities of ProFeat for SASs, but we believe this is a natural approach.

The case study in this paper is of course a highly simplified model of an AUV and its mission. However, we showed that it is feasible to model and analyse a two-layered self-adaptive cyber-physical system as a family of configurations with a controller switching between them. To analyse a real AUV, both the models of the AUV and the environment, and in particular the probabilities, have to be adapted to the robot and the environment with the help of real data and domain experts. We plan to investigate this together with an industrial partner of the MSCA network REMARO (Reliable AI for Marine Robotics).

In the future, we plan to investigate which kind of models can be modelled and analysed as we did with the case study to try to find a general methodology for modelling and analysing SASs as family-based systems. Furthermore, we plan to find optimal strategies for the managing subsystem, i.e., the controller switching between features, e.g., to minimise energy consumption. We would also like to find patterns between choosing a certain feature configuration and the effect of this on quality criteria of the system. Finding such control patterns could help to improve the adaptation logic of the managing subsystem to be more resilient towards faults.

Acknowledgments. We would like to thank Clemens Dubslaff for explaining ProFeat and its usage to us, and for answering numerous questions. Furthermore, we would like to thank Rudolf Schlatte for his help in preparing the artifact for the final artifact submission. This work was supported by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200), by the Italian project NODES (which has received funding from the MUR – M4C2 1.5 of PNRR with grant agreement no. ECS00000036) and by the Italian MUR PRIN 2020TL3X8X project T-LADIES (Typeful Language Adaptation for Dynamic, Interacting and Evolving Systems).

References

1. Araujo, H., Mousavi, M.R., Varshosaz, M.: Testing, Validation, and Verification of Robotic and Autonomous Systems: A Systematic Review. *ACM Transactions on Software Engineering and Methodology* **32**(2), 51:1–51:61 (2023). <https://doi.org/10.1145/3542945>
2. Bencomo, N., Sawyer, P., Blair, G.S., Grace, P.: Dynamically Adaptive Systems are Product Lines too: Using Model-Driven Techniques to Capture Dynamic Variability of Adaptive Systems. In: Thiel, S., Pohl, K. (eds.) *Proceedings of the 12th International Conference on Software Product Lines (SPLC 2008)*. vol. 2, pp. 23–32. Lero, University of Limerick (2008)
3. Brugali, D.: Software Product Line Engineering for Robotics. In: Cavalcanti, A., Dongol, B., Hierons, R., Timmis, J., Woodcock, J. (eds.) *Software Engineering for Robotics*, pp. 1–28. Springer (2021). https://doi.org/10.1007/978-3-030-66494-7_1
4. Brugali, D., Capilla, R., Hinchey, M.: Dynamic Variability Meets Robotics. *IEEE Computer* **48**(12), 94–97 (December 2015). <https://doi.org/10.1109/MC.2015.354>
5. Brugali, D., Hochgeschwender, N.: Managing the Functional Variability of Robotic Perception Systems. In: *Proceedings of the 1st International Conference on Robotic Computing (IRC 2017)*. pp. 277–283. IEEE (2017). <https://doi.org/10.1109/IRC.2017.20>
6. Chrszon, P., Baier, C., Dubslaff, C., Klüppelholz, S.: From Features to Roles. In: *Proceedings of the 24th International Systems and Software Product Line Conference (SPLC 2020)*. pp. 19:1–19:11. ACM (2020). <https://doi.org/10.1145/3382025.3414962>
7. Chrszon, P., Baier, C., Dubslaff, C., Klüppelholz, S.: Interaction detection in configurable systems – A formal approach featuring roles. *Journal of Systems and Software* **196** (2023). <https://doi.org/10.1016/j.jss.2022.111556>
8. Chrszon, P., Dubslaff, C., Klüppelholz, S., Baier, C.: ProFeat: Feature-Oriented Engineering for Family-Based Probabilistic Model Checking. *Formal Aspects of Computing* **30**(1), 45–75 (2018). <https://doi.org/10.1007/s00165-017-0432-4>
9. Cimatti, A., Clarke, E.M., Giunchiglia, E., Giunchiglia, F., Pistore, M., Roveri, M., Sebastiani, R., Tacchella, A.: NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In: Brinksma, E., Larsen, K.G. (eds.) *Proceedings of the 14th International Conference on Computer Aided Verification (CAV 2002)*. LNCS, vol. 2404, pp. 359–364. Springer (2002). https://doi.org/10.1007/3-540-45657-0_29
10. Classen, A., Heymans, P., Schobbens, P.Y., Legay, A., Raskin, J.F.: Model Checking *Lots* of Systems: Efficient Verification of Temporal Properties in Software Product Lines. In: *Proceedings of the 32nd International Conference on Software Engineering (ICSE 2010)*. pp. 335–344. ACM (2010). <https://doi.org/10.1145/1806799.1806850>
11. Dhungana, D., Grünbacher, P., Rabiser, R.: Domain-Specific Adaptations of Product Line Variability Modeling. In: Ralyté, J., Brinkkemper, S., Henderson-Sellers, B. (eds.) *Proceedings of the IFIP WG 8.1 Working Conference on Situational Method Engineering: Fundamentals and Experiences (ME 2007)*. IFIP, vol. 244, pp. 238–251. Springer (2007). https://doi.org/10.1007/978-0-387-73947-2_19
12. Gherardi, L., Brugali, D.: Modeling and Reusing Robotic Software Architectures: the HyperFlex Toolchain. In: *Proceedings of the International Conference on Robotics and Automation (ICRA 2014)*. pp. 6414–6420. IEEE (2014). <https://doi.org/10.1109/ICRA.2014.6907806>

13. Hallsteinsen, S., Hinchey, M., Park, S., Schmid, K.: Dynamic Software Product Lines. In: Capilla, R., Bosch, J., Kang, K.C. (eds.) *Systems and Software Variability Management: Concepts, Tools and Experiences*, pp. 253–260. Springer (2013). https://doi.org/10.1007/978-3-642-36583-6_16
14. Hallsteinsen, S., Stav, E., Solberg, A., Floch, J.: Using Product Line Techniques to Build Adaptive Systems. In: *Proceedings of the 10th International Software Product Line Conference (SPLC 2006)*. pp. 141–150. IEEE (2006). <https://doi.org/10.1109/SPLINE.2006.1691586>
15. Hezavehi, S.M., Weyns, D., Avgeriou, P., Calinescu, R., Mirandola, R., Perez-Palacin, D.: Uncertainty in Self-adaptive Systems: A Research Community Perspective. *ACM Transactions on Autonomous and Adaptive Systems* **15**(4), 10:1–10:36 (2021). <https://doi.org/10.1145/3487921>
16. Hinchey, M., Park, S., Schmid, K.: Building Dynamic Software Product Lines. *IEEE Computer* **45**(10), 22–26 (October 2012). <https://doi.org/10.1109/MC.2012.332>
17. Holzmann, G.J.: *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley (2004)
18. Kephart, J.O., Chess, D.M.: The Vision of Autonomic Computing. *IEEE Computer* **36**(1), 41–50 (January 2003). <https://doi.org/10.1109/MC.2003.1160055>
19. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: Verification of Probabilistic Real-Time Systems. In: Gopalakrishnan, G., Qadeer, S. (eds.) *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV 2011)*. LNCS, vol. 6806, pp. 585–591. Springer (2011). https://doi.org/10.1007/978-3-642-22110-1_47
20. Luckcuck, M., Farrell, M., Dennis, L.A., Dixon, C., Fisher, M.: Formal Specification and Verification of Autonomous Robotic Systems: A Survey. *ACM Computing Surveys* **52**(5), 100:1–100:41 (2019). <https://doi.org/10.1145/3342355>
21. Päckler, J., ter Beek, M.H., Damiani, F., Tapia Tarifa, S.L., Johnsen, E.B.: Formal Modelling and Analysis of a Self-Adaptive Robotic System (Artifact) (August 2023). <https://doi.org/10.5281/zenodo.8275533>
22. Rezende Silva, G., Päckler, J., Zwanepol, J., Alberts, E., Tapia Tarifa, S.L., Gerostathopoulos, I., Johnsen, E.B., Hernández Corbato, C.: SUAVE: An Exemplar for Self-Adaptive Underwater Vehicles. In: *Proceedings of the 18th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2023)*. pp. 181–187. IEEE (2023). <https://doi.org/10.1109/SEAMS59076.2023.00031>
23. Thüm, T., Apel, S., Kästner, C., Schaefer, I., Saake, G.: A Classification and Survey of Analysis Strategies for Software Product Lines. *ACM Computing Surveys* **47**(1), 6:1–6:45 (2014). <https://doi.org/10.1145/2580950>
24. Vandin, A., ter Beek, M.H., Legay, A., Lluch Lafuente, A.: QFLan: A Tool for the Quantitative Analysis of Highly Reconfigurable Systems. In: Havelund, K., Peleska, J., Roscoe, B., de Vink, E. (eds.) *Proceedings of the 22nd International Symposium on Formal Methods (FM 2018)*. LNCS, vol. 10951, pp. 329–337. Springer (2018). https://doi.org/10.1007/978-3-319-95582-7_19
25. Weyns, D.: *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. John Wiley & Sons (2020)
26. Weyns, D., Iftikhar, M.U., de la Iglesia, D.G., Ahmad, T.: A Survey of Formal Methods in Self-Adaptive Systems. In: *Proceedings of the 5th International C* Conference on Computer Science and Software Engineering (C3S2E 2012)*. pp. 67–79. ACM (2012). <https://doi.org/10.1145/2347583.2347592>